

Ex no: 1	Develop basic React components to understand JSX, state, events and routing
Date:	

Aim:

To Develop basic React components to understand JSX, state, events and routing.

1. Question:

Design and implement a Student Dashboard web application using React that demonstrates JSX syntax and element rendering.

The application must be structured using at least five separate functional components, and each component must render meaningful UI using JSX. Use dynamic data, conditional rendering, and props to customize content. The dashboard must include the following:

- A Greeting component that changes the message based on the time of day (e.g., Good Morning, Afternoon, Evening).
- A StudentProfile component that displays student name, roll number, and department using props.
- A CourseList component that dynamically renders a list of enrolled courses.
- A Notification component that conditionally displays alerts if any (based on a boolean prop).
- A Footer component with static contact or institution information.

Use JSX appropriately in all components and maintain a clean, modular file structure. Render all components in a parent App component and display the entire dashboard in a browser.

Code:**App.jsx**

```
import Greeting from './components/Greeting'
import StudentProfile from './components/StudentProfile';
import CourseList from './components/CourseList';
import Notification from './components/Notification';
import Footer from './components/Footer';

function App() {
  const student={ name:'Vasu G',rollNumber:'717823F157',department:'Information Technology'};
  const courses=['MERN Stack','Cloud Computing','Web Application development'];
  const hasAlerts=true;
  return (
    <
      <Greeting/>
      <StudentProfile student={student}/>
      <CourseList courses={courses} />
      <Notification showAlert={hasAlerts}/>
      <Footer/>
    </
  >
  )}
export default App;
```

Greeting.jsx

```
import React from 'react'
export default function Greeting() {
  const hour=new Date().getHours();
  let greeting= 'Hello';
  if(hour<12) greeting='Good Morning';
  else if(hour<18) greeting='Good Afternoon';
  else greeting='Good Evening';
  return <h1>{greeting},Student!</h1>;
}
```

StudentProfile.jsx

```
import React from 'react'
export default function StudentProfile({student}) {
  return (
    <div>
      <h3>Student Profile</h3>
      <ul>
        <li><b>Name:</b>{student.name}</li>
        <li><b>Roll Number:</b>{student.rollNumber}</li>
        <li><b>Department:</b>{student.department}</li>
      </ul>
    </div>
  )
}
```

CourseList.jsx

```
import React from 'react'
export default function CourseList(props) {
  return (
    <div>
      <h3>Enrolled Courses</h3>
      <ul>
        {props.courses.map((course,index)=>(
          <li key={index}>{course}</li>
        ))}
      </ul>
    </div>
  )
}
```

Notification.jsx

```
import React from 'react'
export default function Notification(props) {
  const notification=()=>{
    if(props.showAlert)
      alert("You have pending assignments!");
    else
      alert("No new notifucation");
  }
}
```

```
}  
return <button onClick={notification}>Check Notification</button>  
}
```

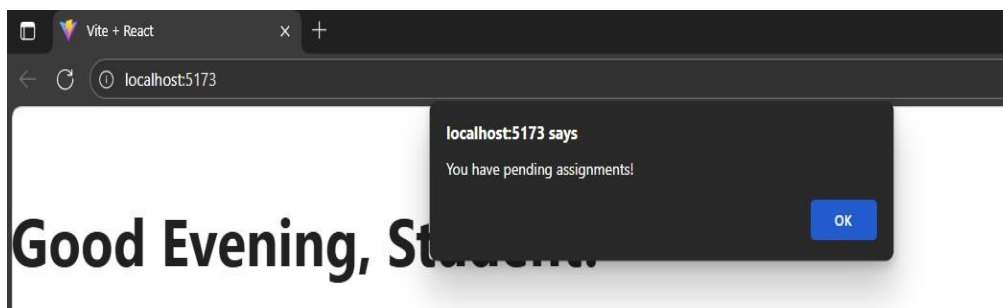
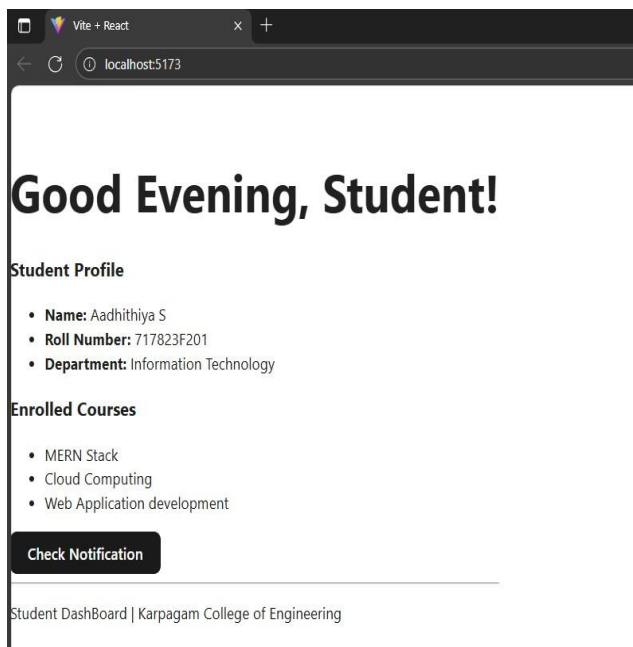
Footer.jsx

import React from 'react'

export default function Footer() {

```
  return (  
    <footer>  
      <hr />  
      <p>Student DashBoard | Karpagam College of Engineering</p>  
    </footer>  
  );  
}
```

Output:



2. Question:

Design and implement a simple Weather Info App using React.

The app should:

- Use the useState hook to manage weather data and loading state.
- Use the useEffect hook to simulate fetching weather data on initial load.
- Use conditional rendering to show a loading message while data is being fetched.
- Use lists and keys to render multiple cities and their weather info.

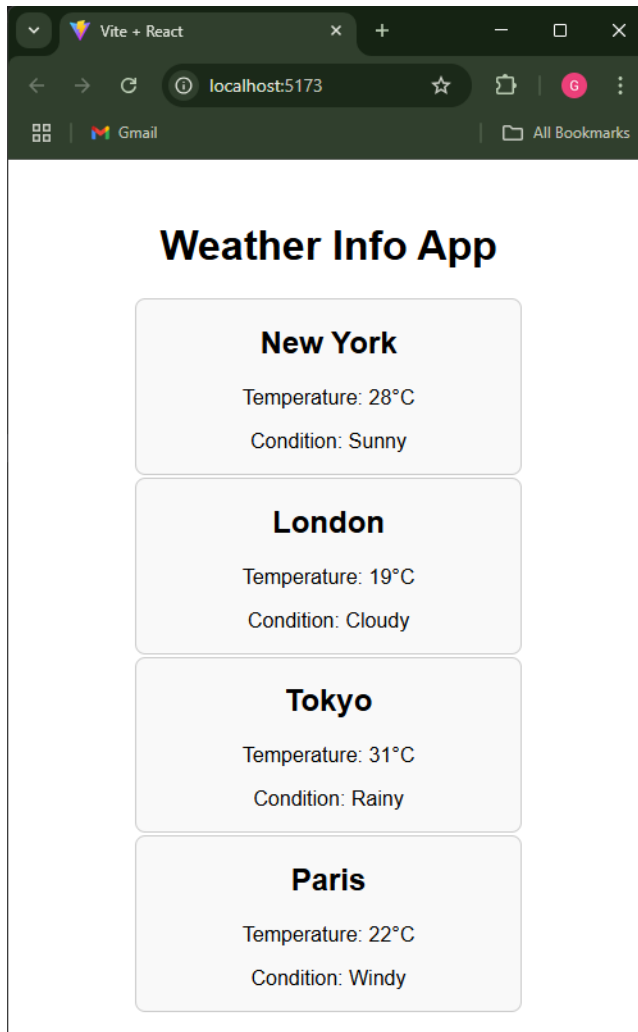
Code:

WeatherInfoApp.jsx

```
import React, { useState, useEffect } from 'react';
export default function WeatherInfoApp () {
  const [weatherData, setWeatherData] = useState([]);
  const [loading, setLoading] = useState(true);
  const sampleweatherdata = [
    { city: 'New York', temperature: 28, condition: 'Sunny' },
    { city: 'London', temperature: 19, condition: 'Cloudy' },
    { city: 'Tokyo', temperature: 31, condition: 'Rainy' },
    { city: 'Paris', temperature: 22, condition: 'Windy' },
  ];
  useEffect(() => {
    setTimeout(() => {
      setWeatherData(sampleweatherdata);
      setLoading(false);
    }, 2000);
  }, []);
  return (
    <div style={styles.container}>
      <h1>Weather Info App</h1>
      {loading ? (<p>Loading weather data...</p>) : (
        <ul style={styles.list}>
          {weatherData.map((weather, index) => (
            <li key={index} style={styles.card}>
              <h2>{weather.city}</h2>
              <p>Temperature: {weather.temperature}°C</p>
              <p>Condition: {weather.condition}</p>
            </li>
          ))}
        </ul>
      )}
    </div>
  );
};
const styles = {
  container: { fontFamily: 'Arial, sans-serif', padding: '20px', textAlign: 'center', },
  list: { listStyleType: 'none', padding: 0 },
  card: { margin: '2px auto', maxWidth: '300px', border: '1px solid #ccc', borderRadius: '8px',
    backgroundColor: '#f9f9f9' }
```

App.jsx

```
import WeatherInfoApp from "../weatherapp/WeatherInfoApp"
function App() {
  return ( <WeatherInfoApp/> )
}
export default App;
```

Output:**3. Question**

Design and implement a Collaborative Team Feedback System using ReactJS.

The app should allow multiple team members to submit feedback and show a live summary. Use controlled components for form input and lift state up to a common parent to aggregate feedback in real-time.

Structure the app using multiple components as follows:

- App: Manages the entire state of the app.
- FeedbackForm: Controlled form component to capture user input.
- FeedbackList: Displays submitted feedback.

- TeamHeader: Shows the total number of submissions dynamically.

All submitted feedback must be stored in a central array in the parent and passed to child components using props.

Code:

App.jsx

```
import { useState } from "react";
import TeamHeader from "../feedback/TeamHeader ";
import FeedbackForm from "../feedback/FeedbackForm";
import FeedbackList from "../feedback/FeedbackList";

function App() {
  const [feedbacks, setFeedbacks] = useState([]);
  const handleFeedbackSubmit = (newFeedback) => {
    setFeedbacks([...feedbacks, newFeedback]);
  };
  return (
    <div>
      <TeamHeader count={feedbacks.length}/>
      <FeedbackForm onSubmit={handleFeedbackSubmit}/>
      <FeedbackList feedbacks={feedbacks}/>
    </div>
  )
}
export default App;
```

TeamHeader.jsx

```
import React from 'react';
export default function TeamHeader({ count }){
  return (
    <div style={{textAlign: 'center',marginBottom: '20px'}}>
      <h1>Team Feedback System</h1>
      <p>Total Submissions: {count}</p>
    </div>
  );
};
```

FeedbackForm.jsx

```
import React, { useState } from 'react';
export default function FeedbackForm({ onSubmit }){
  const [name, setName] = useState("");
  const [message, setMessage] = useState("");
  const handleSubmit = (e) => {
    e.preventDefault();
```

```

    if (!name || !message) return;
    onSubmit({ name, message });
    setName("");
    setMessage("");
  };
  return (
    <form onSubmit={handleSubmit} style={styles.form}>
      <input type="text" placeholder="Your Name" value={name} onChange={(e) =>
setName(e.target.value)} style={styles.input}/>
      <textarea placeholder="Your Feedback" value={message} onChange={(e) =>
setMessage(e.target.value)} style={styles.textarea}/>
      <button type="submit" style={styles.button}>Submit Feedback</button>
    </form>
  );
};
const styles = {
  form: {display: 'flex',flexDirection: 'column',gap: '10px',marginBottom: '20px'},
  input: {padding: '10px',fontSize: '16px'},
  textarea: {padding: '10px',fontSize: '16px',height: '80px'},
  button: {padding: '10px',fontSize: '16px',backgroundColor: '#007bff',color: 'white',border:
'none',cursor: 'pointer'},
};

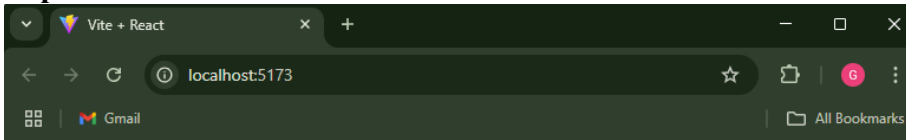
```

FeedbackList.jsx

```

import React from 'react';
export default function FeedbackList({ feedbacks }) {
  return (
    <div>
      <h2>Feedback Submissions</h2>
      <ul style={styles.list}>
        {feedbacks.map((fb, index) => (
          <li key={index} style={styles.card}>
            <strong>{fb.name}</strong> {fb.message}
          </li>
        ))}
      </ul></div>
    );
  };
  const styles = {
    list: { listStyle: 'none', padding: 0 },
    card: { backgroundColor: '#f1f1f1', margin: '10px 0', padding: '10px', borderRadius: '5px' },
  };
};

```

Output:**Team Feedback System**

Total Submissions: 2

Ramu

It is good

Submit Feedback

Feedback Submissions

Vasu: It is very usefull

Raja: It is a good experience

4. Question

Design and implement a News Fetcher React Application that uses Axios to fetch and display live news articles from a public API (e.g., JSONPlaceholder or mock API).

Use the following component structure:

- App: Main state manager, fetches data with Axios.
- NewsList: Receives data as props and renders it.
- NewsItem: Displays individual article (title & body).
- Loader: Shows loading message while fetching.

The app should make the HTTP request when the component mounts and handle both loading and error states gracefully.

Code:**App.jsx**

```
import { useState, useEffect } from "react";
import Loader from "../newsfetcher/Loader";
import NewsList from "../newsfetcher/NewsList";
import axios from 'axios';
function App() {
  const [articles, setArticles] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState("");
  const fetchNews = async () => {
    try {
      const response = await axios.get('https://jsonplaceholder.typicode.com/posts');
```

```

    setArticles(response.data.slice(0, 10));
  } catch (err) {
    setError('Failed to fetch news. Try again later.');
```

} finally {
 setLoading(false);
 }
};
useEffect(() => {
 fetchNews();
}, []);
return (
 <div style={styles.container}>
 <h1>Live News Fetcher</h1>
 {loading && <Loader/>}
 {error && <p style={styles.error}>{error}</p>}
 {!loading && !error && <NewsList articles={articles}/>}
 </div>
)
}
const styles = {
 container: {fontFamily: 'Arial',padding: '20px',maxWidth: '700px',margin: 'auto'},
 error: {color: 'red',textAlign: 'center'},
};
export default App;

Loader.jsx

```

import React from 'react';
export default function Loader(){
  return (
    <div style={styles.loader}>
      <p>Loading news articles...</p>
    </div>
  );
};
const styles = {
  loader: {textAlign: 'center',padding: '20px',fontSize: '18px',fontWeight: 'bold'},
};
```

NewsList.jsx

```

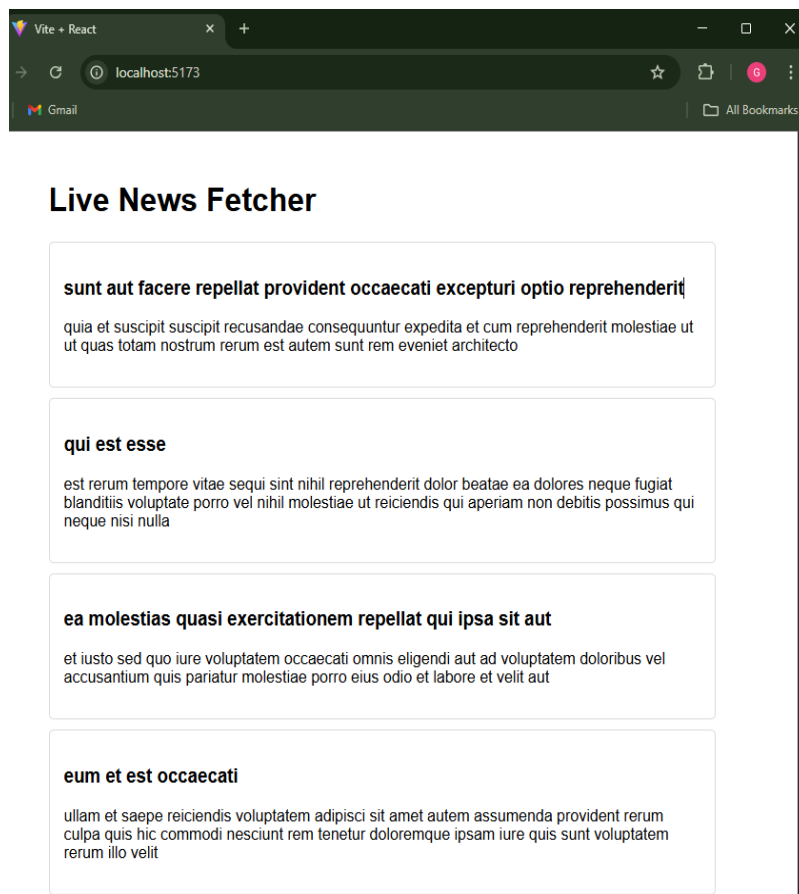
import React from 'react';
import NewsItem from './NewsItem';
export default function NewsList ({ articles }){
  return (
    <div>
      {articles.map((article) => (
        <NewsItem key={article.id} title={article.title} body={article.body} />
      ))}
    </div>
  );
}
```

```

    );
  };
  NewsItem.jsx
  import React from 'react';
  export default function NewsItem({ title, body }){
    return (
      <div style={styles.card}>
        <h3>{title}</h3>
        <p>{body}</p>
      </div>
    );
  };
  const styles = {
    card: {border: '1px solid #ddd',padding: '15px',marginBottom: '10px',borderRadius: '5px',}
  };

```

Output:



Result

Thus, the basic React components to understand JSX, state, events and routing. were implemented successfully.